Calculator using Assembly Language

Department of Computer Engineering

Group Members : -

1. Danish Solkar (24111590805)

2. Kalam Khan (24111590782)

3. Jareer Khan (24111590791)

4. Saud Kazi (24111590790)

1. INTRODUCTION

A calculator is one of the most basic yet important computational tools used in digital systems. Implementing a calculator using assembly language provides a deep understanding of how arithmetic operations are executed at the processor level.

This project focuses on creating a simple calculator program using the **8086-microprocessor instruction set**. Unlike high-level programming languages, assembly language requires direct interaction with registers, memory, and system interrupts. Therefore, this project demonstrates how user input is captured character by character, converted into numeric form, processed, and then converted back into displayable output.

The calculator is menu-driven, meaning the user selects an arithmetic operator, and the program performs the corresponding operation. This makes the program interactive and structured.

2. Algorithm

1) Start the program

2) Initialize data segment

3) Accept the first number

1. Display message: **"Enter First Number"**.
2. Initialize a variable (accumulator) to store the number (initial value = 0).
3. Read characters from keyboard one by one.
4. For each character:
    - If **Enter key (carriage return)** is pressed → stop reading digits.
    - Otherwise:
        - Convert ASCII digit into numeric value (subtract 48).
        - Multiply current number by 10.
        - Add new digit to it.
5. Store the complete multi-digit value as **NUM1**.

4) Accept the operator

1. Display message: **"Enter Operator (+, −, *, /)"**.
2. Read one character from keyboard.
3. Store this character as the selected operator.

5) Accept the second number

1. Display message: **"Enter Second Number"**.
2. Repeat the same procedure used for the first number:
    - Read digits one by one.
    - Convert ASCII to numeric.
    - Multiply previous value by 10.
    - Add new digit.
3. Store the final value as **NUM2**.

6) Decide which operation to perform

Compare the entered operator with valid operators:

1. If operator is **'+'**
    - Load first number into AX.
    - Add second number.
    - Store result.
2. Else if operator is **'−'**
    - Load first number into AX.
    - Subtract second number.
    - Store result.
3. Else if operator is **'*'**
    - Load first number into AX.
    - Multiply by second number.
    - Store lower 16-bit result.

4. Else if operator is **'/'**
    - Clear DX (required before division).
    - Load first number into AX.
    - Divide by second number.
    - Store quotient as result.
5. Else (operator is not valid)
    - Display **"Invalid"** message.
    - Terminate program.

7) Display the result

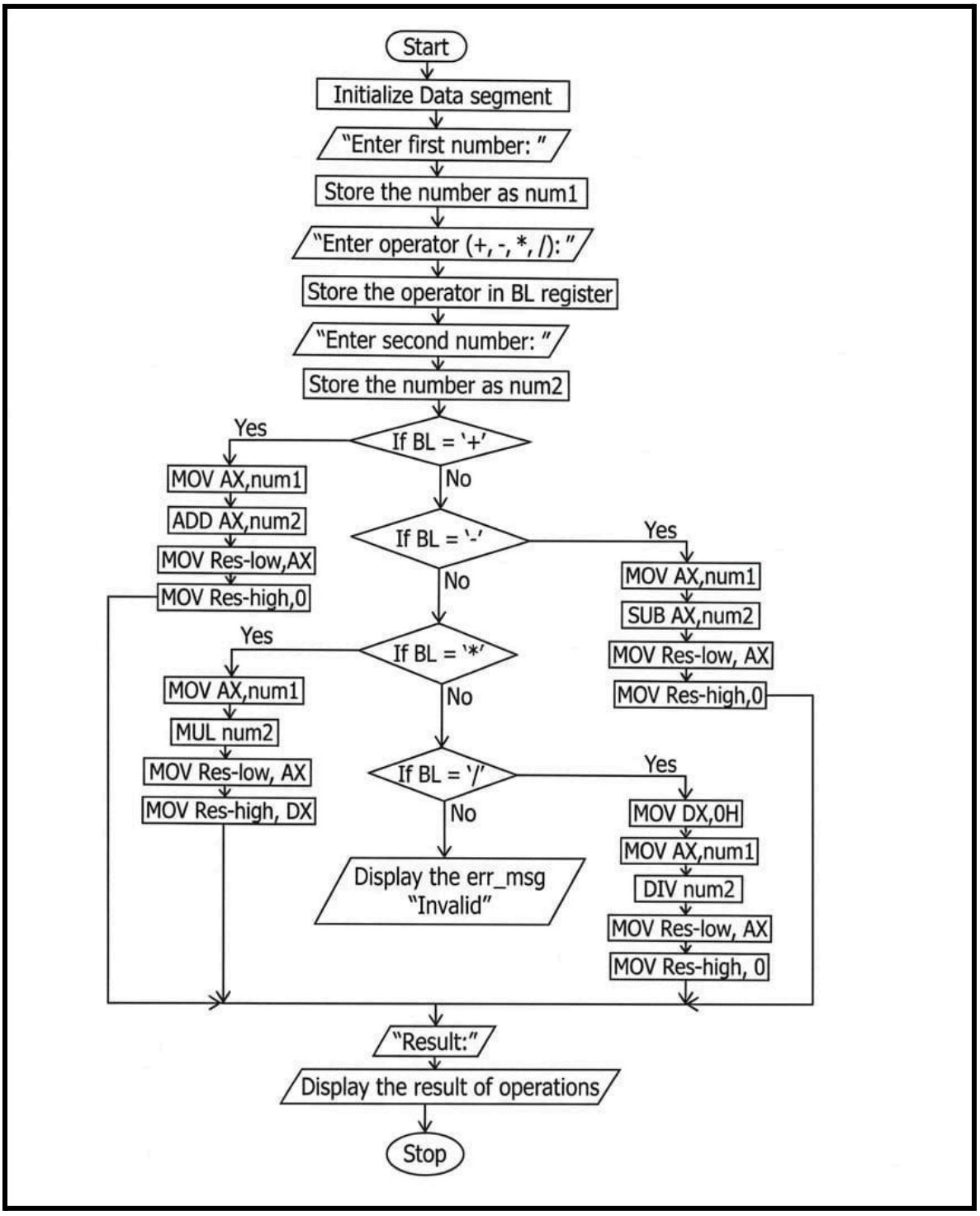
1. Display message: **"Result:"**.
2. Take the numeric result and convert it into digits:
    - Divide number repeatedly by 10.
    - Store remainders (digits) on stack.
3. Pop digits from stack one by one.
4. Convert each digit to ASCII (add 48).
5. Display digits in correct order.

8) Terminate the program

1. Return control to DOS using interrupt.

9) End program

3. Flowchart


Start
Initialize Data segment
"Enter first number: "
Store the number as num1
"Enter operator (+, -, *, /): "
Store the operator in BL register
"Enter second number: "
Store the number as num2
If BL = '+'
Yes
No
MOV AX,num1
ADD AX,num2
MOV Res-low,AX
MOV Res-high,0
If BL = '-'
Yes
No
MOV AX,num1
SUB AX,num2
MOV Res-low, AX
MOV Res-high,0
If BL = '*'
Yes
No
MOV AX,num1
MUL num2
MOV Res-low, AX
MOV Res-high, DX
If BL = '/'
Yes
No
MOV DX,0H
MOV AX,num1
DIV num2
MOV Res-low, AX
MOV Res-high, 0
Display the err_msg
"Invalid"
"Result:"
Display the result of operations
Stop

4. PROGRAM

```
.MODEL SMALL
.STACK 100H

.DATA
   MSG1 DB 10,13,'ENTER FIRST NUMBER: $'
   MSG2 DB 10,13,'ENTER OPERATOR (+, -, *, /): $'
   MSG3 DB 10,13,'ENTER SECOND NUMBER: $'
   MSG4 DB 10,13,'RESULT: $'
   ERR_MSG DB 10,13,'INVALID! $'
   NUM1 DW ?
   NUM2 DW ?
   ; We use two words to store a 32-bit result
   RES_LOW  DW 0
   RES_HIGH DW 0

.CODE
START:
   MOV AX, @DATA
   MOV DS, AX

   ; Get First Number
   LEA DX, MSG1
   MOV AH, 09H
   INT 21H
   CALL READ_NUM
   MOV NUM1, AX

   ; Get Operator
   LEA DX, MSG2
   MOV AH, 09H
   INT 21H
   MOV AH, 01H
```

```
    INT 21H
    MOV BL, AL

    ; Get Second Number
    LEA DX, MSG3
    MOV AH, 09H
    INT 21H
    CALL READ_NUM
    MOV NUM2, AX

    ; Decision Logic
    CMP BL, '+'
    JE  DO_ADD
    CMP BL, '-'
    JE  DO_SUB
    CMP BL, '*'
    JE  DO_MUL
    CMP BL, '/'
    JE  DO_DIV
    JMP ERROR_EXIT

DO_ADD:
    MOV AX, NUM1
    ADD AX, NUM2
    MOV RES_LOW, AX
    MOV RES_HIGH, 0    ; No overflow possible in 16-bit add for small nums
    JMP SHOW_RESULT

DO_SUB:
    MOV AX, NUM1
    SUB AX, NUM2
    MOV RES_LOW, AX
    MOV RES_HIGH, 0
    JMP SHOW_RESULT
```

```
DO_MUL:
   MOV AX, NUM1
   MUL NUM2          ; DX:AX = AX * NUM2
   MOV RES_LOW, AX    ; Lower 16 bits
   MOV RES_HIGH, DX   ; Upper 16 bits (Fixes the 9578 error)
   JMP SHOW_RESULT

DO_DIV:
   MOV DX, 0
   MOV AX, NUM1
   DIV NUM2
   MOV RES_LOW, AX
   MOV RES_HIGH, 0
   JMP SHOW_RESULT

ERROR_EXIT:
   LEA DX, ERR_MSG
   MOV AH, 09H
   INT 21H
   JMP FINISH

SHOW_RESULT:
   LEA DX, MSG4
   MOV AH, 09H
   INT 21H
   CALL WRITE_NUM_32  ; Call the updated 32-bit print routine

FINISH:
   MOV AH, 4CH
   INT 21H

; --- PROCEDURE: READ MULTI-DIGIT NUMBER ---
READ_NUM PROC
```

```
    PUSH BX
    PUSH CX
    MOV BX, 0
READ_LOOP:
    MOV AH, 01H
    INT 21H
    CMP AL, 13
    JE READ_DONE
    SUB AL, 48
    MOV CL, AL
    MOV CH, 0
    MOV AX, 10
    MUL BX
    ADD AX, CX
    MOV BX, AX
    JMP READ_LOOP
READ_DONE:
    MOV AX, BX
    POP CX
    POP BX
    RET
READ_NUM ENDP

; --- PROCEDURE: WRITE 32-BIT NUMBER (DX:AX) ---
WRITE_NUM_32 PROC
    PUSH AX
    PUSH BX
    PUSH CX
    PUSH DX

    MOV AX, RES_LOW
    MOV DX, RES_HIGH
    MOV BX, 10
    MOV CX, 0
```

```
CONVERT_LOOP:
   ; Standard 32-bit division logic for 8086
   ; To avoid overflow, we divide the high part, then the low part
   PUSH AX          ; Save low part
   MOV AX, DX       ; Move high part to AX
   MOV DX, 0        ; Clear DX
   DIV BX           ; DX:AX / 10 -> AX = High Quotient, DX = High Remainder
   MOV BP, AX       ; Store High Quotient in BP
   POP AX           ; Restore low part
   DIV BX           ; DX:AX / 10 -> AX = Low Quotient, DX = Real Remainder

   PUSH DX          ; Save the actual digit
   INC CX
   MOV DX, BP       ; Put High Quotient back into DX for next loop

   OR AX, DX        ; Check if both DX and AX are 0
   JNZ CONVERT_LOOP

DISPLAY_LOOP:
   POP DX
   ADD DL, 48
   MOV AH, 02H
   INT 21H
   LOOP DISPLAY_LOOP

   POP DX
   POP CX
   POP BX
   POP AX
   RET
WRITE_NUM_32 ENDP

END START
```

5. OUTPUT OF THE PROGRAM

```
C:\>cal.exe

ENTER FIRST NUMBER: 100

ENTER OPERATOR (+, -, *, /): +
ENTER SECOND NUMBER: 200

RESULT: 300
```

```
C:\>cal.exe

ENTER FIRST NUMBER: 256

ENTER OPERATOR (+, -, *, /): -
ENTER SECOND NUMBER: 78

RESULT: 178
```

```
C:\>cal.exe

ENTER FIRST NUMBER: 123

ENTER OPERATOR (+, -, *, /): *
ENTER SECOND NUMBER: 456

RESULT: 56088
```

```
C:\>cal.exe

ENTER FIRST NUMBER: 12500

ENTER OPERATOR (+, -, *, /): /
ENTER SECOND NUMBER: 5

RESULT: 2500
```

6. APPLICATIONS

1. Educational Learning Tool
Helps students understand low-level programming, microprocessor operations, and assembly language structure.

2. Understanding Number Conversion Techniques
Demonstrates conversion between ASCII and numeric formats.

3. Foundation for Embedded Systems Programming
Many embedded systems require direct hardware control similar to assembly programming.

4. Debugging and Performance Optimization
Learning assembly helps programmers understand how high-level programs are executed internally.

5. Basic Arithmetic Processing in Low-Memory Systems
Useful where minimal resources are available.

7. CONCLUSION

The menu-driven calculator implemented using 8086 assembly language successfully performs basic arithmetic operations on multi-digit numbers. The project demonstrates essential microprocessor programming concepts such as register manipulation, interrupt handling, procedure calls, control flow, and data conversion. It highlights how simple operations in high-level languages require multiple low-level steps when implemented directly on hardware.

Overall, this project strengthens understanding of processor-level computation and structured assembly programming.

8. REFERENCES

1. Douglas V. Hall — *Microprocessors and Interfacing: Programming and Hardware*.
2. Kip R. Irvine — *Assembly Language for x86 Processors*.
3. Intel 8086 Microprocessor Programmer's Reference Manual by Intel.
4. Ramesh S. Gaonkar — *Microprocessor Architecture, Programming, and Applications*.